

Unidade 4

Aplicações com Python

Aula 1

Introdução à Programação Web com Python

Introdução à programação web com Python

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

Python é uma linguagem de programação amplamente utilizada no desenvolvimento web, segmento que envolve a criação de aplicativos e sites para a internet. Esse processo abrange tanto o lado do cliente (front-end) quanto o lado do servidor (back-end), utilizando linguagens como HTML, CSS e

JavaScript, além de frameworks como React, Django ou Flask. Nesse contexto, a linguagem Python age como um “mediador” dessas interações.

O front-end é a interface visível para os usuários. Desenvolvedores front-end utilizam HTML, CSS e JavaScript, frequentemente integrando frameworks como React, Angular ou Vue.js para criar interatividade. A Python também é utilizada, por exemplo, em projetos que envolvem a construção de interfaces usando Dash ou Flask.

O back-end é responsável pela lógica, processamento e armazenamento de dados. Desenvolvedores back-end usam linguagens como Python, Java ou Node.js, além de frameworks como Django, Flask ou Express, para criar a lógica do servidor e gerenciar interações com o banco de dados.

Vamos imaginar que você esteja participando de um curso de desenvolvimento web e sua primeira tarefa seja criar uma página de perfil pessoal.

Vamos Começar!

Visão geral sobre o desenvolvimento de sistemas para a web

O desenvolvimento de sistemas para a web com Python se apresenta como uma jornada empolgante, que combina a versatilidade dessa linguagem de programação com as demandas dinâmicas da internet moderna. A linguagem Python, conhecida por sua legibilidade, facilidade de aprendizado e vasta comunidade de desenvolvedores, tem se destacado tanto no front-end quanto no back-end, oferecendo soluções abrangentes para a criação de aplicativos web robustos e eficientes.

No início, as páginas web eram estáticas e limitadas em sua interatividade. Com o tempo, a necessidade de interfaces mais dinâmicas e funcionais levou ao desenvolvimento de linguagens e frameworks voltados para a web. A Python entrou nesse cenário, inicialmente no back-end, ganhando destaque com frameworks como Django e Flask.

O desenvolvimento web é dividido principalmente em duas partes: front-end e back-end. O front-end lida com a interface do usuário, enquanto o back-end gerencia a lógica, o processamento de dados e a interação com o banco de dados. A linguagem Python é versátil o suficiente para ser utilizada em ambos os contextos (front-end e back-end), proporcionando uma integração suave e eficiente entre essas camadas.

A Python possui uma gama de frameworks e bibliotecas dedicados ao desenvolvimento web. Django, um framework full-stack, é reconhecido pela sua robustez e convenções claras. Flask, uma estrutura mais leve, é apreciada pela sua flexibilidade. Ambos os recursos são excelentes escolhas para o back-end. Além disso, bibliotecas front-end como React, Vue.js e Angular podem ser facilmente integradas à linguagem Python.

O desenvolvimento web moderno destaca a importância das APIs (Interfaces de Programação de Aplicações) para otimizar a comunicação entre o front-end e o back-end. A Python, com frameworks como FastAPI e Django Rest Framework, oferece soluções poderosas para criar APIs escaláveis e bem documentadas.

A tecnologia web está em constante evolução. O uso de arquiteturas sem servidor, microserviços, containers e orquestradores como Kubernetes tornou-se comum. Python, com suas características de legibilidade e flexibilidade, se integra facilmente a essas tecnologias, permitindo que os desenvolvedores permaneçam atualizados com as últimas tendências dessa área.

O desenvolvimento web com Python não está isento de desafios. A escalabilidade, a segurança e a necessidade de interfaces mais ricas são considerações constantes. No entanto, esses desafios também proporcionam oportunidades para aprimorar habilidades e explorar soluções inovadoras.

Em resumo, o desenvolvimento de sistemas para a web com Python viabiliza uma abordagem completa e poderosa. Seja construindo uma aplicação web simples ou um sistema complexo, a linguagem Python proporciona ferramentas e recursos para enfrentar os desafios do desenvolvimento web moderno.

Com uma comunidade ativa e uma série de recursos disponíveis, a Python continua a ser uma escolha excelente para aqueles que querem mergulhar no mundo dinâmico do desenvolvimento web.

Front-end e back-end

O front-end de uma aplicação web é a interface com a qual os usuários interagem diretamente. A partir de agora, vamos descobrir como criar uma experiência front-end simples usando Python no Google Colab.

O HTML é a espinha dorsal do conteúdo na web. Podemos criar elementos HTML usando Python no Colab e, em seguida, incorporá-los em nossa página.

Vamos criar uma página HTML básica:

```
# Criando uma página HTML usando Python
html_code =
<!DOCTYPE html>
<html>
<head>
  <title>Exemplo de Front-end com Python</title>
</head>
```

```
<body>
  <h1>Olá, mundo!</h1>
  <p>Esta é uma página web criada usando Python no Google Colab.</p>
</body>
</html>
```

```
# Exibindo a página HTML
from IPython.display import HTML
HTML(html_code)
Olá, mundo!
Esta é uma página web criada usando Python no Google Colab.
```

Nesse exemplo, geramos uma página HTML usando uma string Python e a exibimos no Colab. Isso demonstra como podemos incorporar HTML no Colab para criar conteúdo front-end.

O back-end de uma aplicação web lida com a lógica, o processamento de dados e a interação com o servidor. Vamos entender, então, como criar um back-end simples usando Python no Google Colab.

O Flask é um framework web leve para Python. Embora não seja a escolha ideal para ambientes de produção no Colab, podemos usá-lo para criar um exemplo básico de servidor back-end:

```
!ngrok authtoken 2YE2L21rdpK3IO8oTpHoTCL9z7h_xxsYeAHKwXUzQjWLZduY
# usar esse código de autenticação
# Em outra célula rodar:
!pip install flask flask-ngrok

from flask import Flask
from flask_ngrok import run_with_ngrok

app = Flask(__name__)
run_with_ngrok(app)

@app.route('/')
def index():
    return 'Olá, esta é a rota principal do back-end!'

if __name__ == '__main__':
    app.run()
* Serving Flask app '__main__'
* Debug mode: off
```

```
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
```

```
* Running on http://127.0.0.1:5000
```

```
INFO:werkzeug:Press CTRL+C to quit
```

```
* Running on http://e399-34-141-146-10.ngrok-free.app #muda quando rodamos de novo
```

```
* Traffic stats available on http://127.0.0.1:4040
```

A célula apresentada anteriormente cria um aplicativo Flask e o expõe publicamente usando o ngrok para criar um túnel. Isso permite que você acesse seu servidor Flask temporário a partir de um link gerado pelo ngrok.

Esses exemplos ilustram a integração do front-end e do back-end usando Python no Google Colab. Para projetos mais complexos, considere o uso de ambientes de desenvolvimento web dedicados, mas, para experimentação e demonstração, o Colab dispõe uma plataforma interativa e prática.

Siga em Frente...

Primeiras páginas web

Vamos, agora, melhorar nossa página feita com HTML no Colab!

```
# Exemplo de HTML com botão usando Python
html_code =
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Minha Primeira Página Web</title>
  <style>
    body {
      font-family: 'Arial', sans-serif;
      background-color: #f8f8f8;
      margin: 0;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
```

```
}

.container {
  text-align: center;
  padding: 40px;
  background-color: #fff;
  border-radius: 8px;
  box-shadow: 0 0 20px rgba(0, 0, 0, 0.2);
}

h1 {
  color: #3498db;
  font-size: 2em;
  margin-bottom: 20px;
}

p {
  color: #555;
  font-size: 1.2em;
}

button {
  background-color: #3498db;
  color: #fff;
  font-size: 1.2em;
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  transition: background-color 0.3s ease;
}

button:hover {
  background-color: #2980b9;
}
</style>
</head>
<body>
  <div class="container">
    <h1>Olá, Mundo!</h1>
    <p>Esta é minha primeira página web criada com Python no Colab. Bem-vindo ao mundo
da programação web!</p>
    <button onclick="alert('Botão clicado!')">Clique em Mim</button>
  </div>
```

```
</body>
</html>

# Exibindo a página HTML
from IPython.display import HTML
HTML(html_code)
```

Alteramos as cores de fundo, texto e sombra para tornar a página mais atraente. Ajustamos o tamanho da fonte e o espaçamento para melhorar a legibilidade. Centralizamos os elementos na tela usando o Flexbox. Adicionamos uma sombra sutil e bordas arredondadas ao container para um visual mais moderno. Adicionamos um botão com um estilo básico. O botão aciona um alerta simples quando clicado.

Vamos Exercitar?

Vamos pensar no problema apresentado no início desta aula. Imagine que você esteja participando de um curso de desenvolvimento web e sua primeira tarefa seja criar uma página de perfil pessoal.

```
from IPython.display import HTML
html_code =
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Meu Perfil</title>
</head>
<body style="font-family: 'Arial', sans-serif; background-color: #f8f8f8; margin: 0; padding: 0;">

  <header style="text-align: center; background-color: #3498db; color: #fff; padding: 20px;">
    <h1 style="margin: 0;">Anderson Inácio Salata de Abreu</h1>
    <p style="margin: 5px 0;">Desenvolvedor Web</p>
  </header>

  <section style="margin: 20px; text-align: center;">
    
    <div id="informacoes-pessoais" style="max-width: 400px; margin: 0 auto;">
      <p>Cidade: Sorocaba </p>
      <p>País: Brasil</p>
```

```
<p>Interesses: Web Development, Programação, etc.</p>
</div>
</section>

<section style="margin: 20px; text-align: center;">
  <h2>Habilidades</h2>
  <ul style="list-style: none; padding: 0;">
    <li>Linguagens: Python, HTML, CSS, JavaScript</li>
    <li>Ferramentas: Git, VS Code</li>
  </ul>
</section>

<section style="margin: 20px; text-align: center;">
  <h2>Projeto Recente</h2>
  <p>Trabalhando em um site pessoal para mostrar meu portfólio.</p>
</section>

<footer style="text-align: center; margin-top: 20px;">
  <a href="https://www.linkedin.com/in/andersonsalata/" target="_blank" style="margin: 0
10px; color: #3498db; text-decoration: none;">LinkedIn</a>
</footer>
</body>
</html>

# Exibindo a página HTML
HTML(html_code)
```

Nesse código, criamos um site com algumas informações pessoais utilizando HTML. Além disso, como “mediador”, usamos o Python no Colab!

Espero que você tenha gostado da solução! Lembre-se de que a prática é importante! Mude alguma parte do código e diversifique seu conhecimento!

Saiba mais

1. O Jupyter Notebook fornece um ambiente no qual você pode trabalhar com facilidade o seu código na linguagem Python. Para saber mais detalhes, visite: [jupyter](https://jupyter.org/).
2. Como mencionado anteriormente, uma leitura interessante para quem está começando a programar em Python é a do livro [Python 3: conceitos e aplicações: uma abordagem didática](#).

BANIN, S. L. **Python 3**: conceitos e aplicações: uma abordagem didática. São Paulo: Érica, 2018. E-book.

3. O Django, enquanto framework, se destaca como uma ferramenta consolidada, simplificando significativamente o processo de criação de aplicações web. Sua vantagem reside na capacidade de facilitar o desenvolvimento sem depender de um ambiente de execução robusto, concedendo, ao mesmo tempo, a flexibilidade de integração rápida com servidores compatíveis para a distribuição de aplicações de modo eficiente e descomplicado.

MACIEL, F. M. B. [Python e Django: desenvolvimento web moderno e ágil](#). Rio de Janeiro: Alta Book, 2018. E-book.

Referências

BANIN, S. L. **Python 3**: conceitos e aplicações: uma abordagem didática. São Paulo: Érica, 2018. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/books/9788536530253>. Acesso em: 21 out. 2023.

MACIEL, F. M. B. **Python e Django**: desenvolvimento web moderno e ágil. Rio de Janeiro: Alta Book, 2018. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555200973>. Acesso em: 21 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos**: lógica para desenvolvimento de programação de computadores. 29. ed. São Paulo: Érica, 2019.

TERUEL, E. C. **HTML 5**: guia prático. São Paulo: Érica, 2013. E-book.

TRY JUPYTER. **Jupyter**, [s. d.]. Disponível em: <https://jupyter.org/try>. Acesso em: 15 nov. 2023.

Aula 2

Introdução a Programação Mobile com Python

Introdução a Programação Mobile com Python

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

O desenvolvimento móvel revoluciona a maneira como interagimos com a tecnologia, oportunizando a criação de aplicativos que transformam nossos dispositivos em ferramentas poderosas.

O KivyMD emerge como um recurso robusto para o desenvolvimento móvel em Python, oferecendo uma gama diversificada de widgets e mecanismos inspirados no Material Design. Essa biblioteca dinâmica capacita desenvolvedores para a construção de aplicativos móveis visualmente atraentes, mantendo a flexibilidade e a eficiência.

Ao adotar o MDTabs no KivyMD, a organização estruturada de interfaces móveis ganha destaque. A utilização inteligente de widgets e abas simplifica a navegação do usuário, fornecendo uma base sólida para elaborar aplicações móveis intuitivas e eficazes.

Para contextualizar sua aprendizagem, imagine a seguinte situação: você foi designado para desenvolver uma calculadora. A princípio, você deve replicar uma versão comum, mas, posteriormente, precisará modificá-la e criar novas funcionalidades.

Vamos Começar!

Introdução ao desenvolvimento mobile

O desenvolvimento mobile refere-se à criação de aplicativos para dispositivos móveis, como smartphones e tablets. Enquanto muitos desenvolvedores associam o desenvolvimento mobile a linguagens nativas específicas de plataformas – como Swift, para iOS, e Kotlin/Java, para

Android –, o uso de Python tem ganhado popularidade por causa de sua versatilidade e facilidade de aprendizado.

A linguagem Python, com sua sintaxe clara e concisa, oferece uma abordagem diferenciada para o desenvolvimento mobile. Frameworks notáveis, como Kivy, BeeWare e o recente Flutter com suporte para Python, permitem aos desenvolvedores criar aplicativos atraentes e funcionais usando a Python como linguagem principal.

O Kivy é uma opção poderosa para o desenvolvimento mobile em Python, pois apresenta uma estrutura robusta para construir interfaces gráficas multitouch. Com suporte para várias plataformas, incluindo Android e iOS, o Kivy simplifica o processo de criação de aplicativos visualmente impressionantes, promovendo a reutilização de código entre diferentes sistemas operacionais.

Ao optar pela Python no desenvolvimento mobile, os desenvolvedores podem aproveitar os benefícios de uma comunidade ativa, uma vasta biblioteca de módulos e a agilidade proporcionada pela linguagem. No entanto, é importante estar ciente dos prováveis desafios, como o desempenho comparado a linguagens nativas e possíveis limitações de acesso a recursos específicos do dispositivo.

O uso de Python no desenvolvimento mobile abre portas para desenvolvedores que já estão familiarizados com a linguagem, possibilitando que eles ampliem seu alcance para o universo mobile. Com frameworks robustos disponíveis, a escolha entre Python e outras linguagens para desenvolvimento mobile dependerá dos requisitos específicos do projeto e das preferências da equipe de desenvolvimento.

Siga em Frente...

KivyMD

O KivyMD é uma extensão do Kivy, um framework Python para o desenvolvimento de aplicativos multitouch. O KivyMD estende as capacidades do Kivy ao integrar os princípios de design do Material Design do Google, concedendo uma experiência visualmente atraente e consistente em dispositivos móveis.

As principais características do KivyMD são:

1. **Material Design:** o KivyMD adota os padrões de design do Material Design, oferecendo uma interface de usuário moderna, consistente e intuitiva, a qual inclui elementos como botões flutuantes, barras de navegação e efeitos de transição.
2. **Componentes prontos para uso:** o framework fornece uma variedade de componentes prontos para uso, como botões, caixas de diálogo, cartões e muitos outros recursos. Esses

componentes seguem as diretrizes do Material Design, otimizando a criação de interfaces visualmente atraentes.

3. **Suporte a múltiplas plataformas:** assim como o Kivy, o KivyMD é multiplataforma, permitindo que os aplicativos sejam executados em diferentes sistemas operacionais, incluindo Android, iOS, Windows, Linux e macOS.

Widgets e organização de telas utilizando o MDTabs

Os widgets desempenham um papel fundamental no desenvolvimento de interfaces de usuário interativas. No contexto do KivyMD (Material Design Components for Kivy), os widgets são blocos de construção essenciais para criar aplicativos visualmente atraentes.

O MDTabs é um widget poderoso do KivyMD projetado para facilitar a organização de conteúdo em diferentes abas. Isso é particularmente útil quando você precisa exibir várias seções ou funcionalidades em um aplicativo sem sobrecarregar a tela principal.

```
# Instalação do KivyMD
!pip install kivymd
from kivy.app import App
from kivy.lang import Builder
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.label import Label
from kivy.uix.tabbedpanel import TabbedPanel, TabbedPanelItem

Builder.load_string("""
<TabsLayout>:
    TabbedPanel:
        do_default_tab: False
        TabbedPanelItem:
            text: 'Tab 1'
            BoxLayout:
                orientation: 'vertical'
                Label:
                    text: 'Content for Tab 1'
        TabbedPanelItem:
            text: 'Tab 2'
            BoxLayout:
                orientation: 'vertical'
                Label:
                    text: 'Content for Tab 2'
""")

class TabsLayout(BoxLayout):
    pass
```

```
class TabsApp(App):
    def build(self):
        return TabsLayout()

if __name__ == '__main__':
    TabsApp().run()
```

Esse é um exemplo básico de um aplicativo Kivy com guias. O conteúdo de cada guia é um simples rótulo, mas você pode personalizá-lo e expandi-lo conforme necessário, para atender aos requisitos do seu aplicativo. **Esse código foi rodado utilizando-se o Jupyter Notebook. O Google Colab não consegue gerar o aplicativo!**

Vamos Exercitar?

Vamos pensar no problema apresentado no início desta aula. Devemos criar o aplicativo de uma calculadora! Nesse caso, vamos usar novamente o **Jupyter Notebook**.

```
!pip install kivymd
from kivy.lang import Builder
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.gridlayout import GridLayout
from kivymd.app import MDApp
from kivymd.uix.button import MDRaisedButton
from kivymd.uix.textfield import MDTextField
from kivy.metrics import dp
KV = """
<CalculatorApp>:
    orientation: 'vertical'
    MDTextField:
        id: input_field
        hint_text: "Insira um número"
        helper_text_mode: "on_focus"
        input_filter: "float"
    GridLayout:
        cols: 4
        spacing: dp(10)
        MDRaisedButton:
            text: "1"
            on_press: app.on_number_press(1)
        MDRaisedButton:
            text: "2"
            on_press: app.on_number_press(2)
```

```
MDRaisedButton:
    text: "3"
    on_press: app.on_number_press(3)
MDRaisedButton:
    text: "+"
    on_press: app.on_operator_press("+")
MDRaisedButton:
    text: "4"
    on_press: app.on_number_press(4)
MDRaisedButton:
    text: "5"
    on_press: app.on_number_press(5)
MDRaisedButton:
    text: "6"
    on_press: app.on_number_press(6)
MDRaisedButton:
    text: "-"
    on_press: app.on_operator_press("-")
MDRaisedButton:
    text: "7"
    on_press: app.on_number_press(7)
MDRaisedButton:
    text: "8"
    on_press: app.on_number_press(8)
MDRaisedButton:
    text: "9"
    on_press: app.on_number_press(9)
MDRaisedButton:
    text: "*"
    on_press: app.on_operator_press("*")
MDRaisedButton:
    text: "C"
    on_press: app.clear_input()
MDRaisedButton:
    text: "0"
    on_press: app.on_number_press(0)
MDRaisedButton:
    text: "="
    on_press: app.calculate_result()
MDRaisedButton:
    text: "/"
    on_press: app.on_operator_press("/")
'''
```

```
class CalculatorApp(BoxLayout):
```

```
def on_number_press(self, number):
    current_text = self.ids.input_field.text
    new_text = f{current_text}{number}
    self.ids.input_field.text = new_text
def on_operator_press(self, operator):
    current_text = self.ids.input_field.text
    new_text = f{current_text} {operator} "
    self.ids.input_field.text = new_text
def clear_input(self):
    self.ids.input_field.text =
def calculate_result(self):
    try:
        result = eval(self.ids.input_field.text)
        self.ids.input_field.text = str(result)
    except Exception as e:
        self.ids.input_field.text =
class CalculatorMDApp(MDApp):
    def build(self):
        return CalculatorApp()
    def on_number_press(self, number):
        self.root.on_number_press(number)
    def on_operator_press(self, operator):
        self.root.on_operator_press(operator)
    def clear_input(self):
        self.root.clear_input()
    def calculate_result(self):
        self.root.calculate_result()
if __name__ == :
    Builder.load_string(KV)
    CalculatorMDApp().run()
```

Por meio desse código, criamos uma calculadora! Maneiro, não é? Agora, faça modificações e deixe a calculadora da forma que você achar melhor.

Espero que você tenha gostado da solução! Lembre-se de que a prática é importante! Mude alguma parte do código e diversifique seu conhecimento!

!pip install kivymd

from kivy.lang import Builder

from kivy.uix.boxlayout import BoxLayout

from kivy.uix.gridlayout import GridLayout

from kivymd.app import MDApp

```
from kivymd.uix.button import MDRaisedButton
```

```
from kivymd.uix.textfield import MDTextField
```

```
from kivy.metrics import dp
```

```
KV = """
```

```
<CalculatorApp>:
```

```
    orientation: 'vertical'
```

```
    MDTextField:
```

```
        id: input_field
```

```
        hint_text: "Insira um número"
```

```
        helper_text_mode: "on_focus"
```

```
        input_filter: "float"
```

```
    GridLayout:
```

```
        cols: 4
```

```
        spacing: dp(10)
```

```
        MDRaisedButton:
```

```
            text: "1"
```

```
            on_press: app.on_number_press(1)
```

```
        MDRaisedButton:
```

```
            text: "2"
```

```
            on_press: app.on_number_press(2)
```

```
        MDRaisedButton:
```

```
            text: "3"
```

```
on_press: app.on_number_press(3)
```

MDRaisedButton:

```
text: "+"
```

```
on_press: app.on_operator_press("+")
```

MDRaisedButton:

```
text: "4"
```

```
on_press: app.on_number_press(4)
```

MDRaisedButton:

```
text: "5"
```

```
on_press: app.on_number_press(5)
```

MDRaisedButton:

```
text: "6"
```

```
on_press: app.on_number_press(6)
```

MDRaisedButton:

```
text: "-"
```

```
on_press: app.on_operator_press("-")
```

MDRaisedButton:

```
text: "7"
```

```
on_press: app.on_number_press(7)
```

MDRaisedButton:

```
text: "8"
```

```
on_press: app.on_number_press(8)
```

MDRaisedButton:

text: "9"

on_press: app.on_number_press(9)

MDRaisedButton:

text: "*"

on_press: app.on_operator_press("*")

MDRaisedButton:

text: "C"

on_press: app.clear_input()

MDRaisedButton:

text: "0"

on_press: app.on_number_press(0)

MDRaisedButton:

text: "="

on_press: app.calculate_result()

MDRaisedButton:

text: "/"

on_press: app.on_operator_press("/")

'''

class CalculatorApp(BoxLayout):

def on_number_press(self, number):

current_text = self.ids.input_field.text

```
        self.ids.input_field.text = new_text

def on_operator_press(self, operator):

    current_text = self.ids.input_field.text

    self.ids.input_field.text = new_text

def clear_input(self):

def calculate_result(self):

    try:

        result = eval(self.ids.input_field.text)

        self.ids.input_field.text = str(result)

    except Exception as e:

class CalculatorMDApp(MDApp):

    def build(self):

        return CalculatorApp()

    def on_number_press(self, number):

        self.root.on_number_press(number)

    def on_operator_press(self, operator):

        self.root.on_operator_press(operator)

    def clear_input(self):

        self.root.clear_input()

    def calculate_result(self):

        self.root.calculate_result()

Builder.load_string(KV)
```

```
CalculatorMDApp().run()
```

Por meio desse código, criamos uma calculadora! Maneiro, não é? Agora, faça modificações e deixe a calculadora da forma que você achar melhor.

Espero que você tenha gostado da solução! Lembre-se de que a prática é importante! Mude alguma parte do código e diversifique seu conhecimento!

Saiba mais

Kivy é uma biblioteca usada para o desenvolvimento de software com código-fonte aberto, voltado ao rápido desenvolvimento de aplicações que utilizam novas interfaces de usuário. Para saber mais detalhes, visite: <https://python.org.br/mobile/>.

Como mencionado anteriormente, uma leitura interessante para quem está começando a programar em Python é a do livro *Python 3: conceitos e aplicações: uma abordagem didática*.

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book.

<https://integrada.minhabiblioteca.com.br/#/books/9788536530253>

Também encorajo você a conhecer o livro *Fundamentos de desenvolvimento mobile*, que apresenta conceitos e aplicações para mobile.

MORAIS, M. S. de F. *et al.* **Fundamentos de desenvolvimento mobile**. Porto Alegre: Grupo A, 2022.

<https://integrada.minhabiblioteca.com.br/#/books/9786556903057>

Referências

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/books/9788536530253>. Acesso em: 21 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29. ed. São Paulo: Érica, 2019.

MORAIS, M. S. de F. *et al.* **Fundamentos de desenvolvimento mobile**. Porto Alegre: Grupo A, 2022. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786556903057>. Acesso em: 21 out. 2023.

PYTHON para desenvolvimento mobile. **Python Brasil**, 3 jun. 2016. Disponível em: <https://python.org.br/mobile/>. Acesso em: 15 nov. 2023.

Aula 3

Testes com Python

Testes com Python

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

Os testes em Python desempenham um papel fundamental no desenvolvimento de softwares, garantindo que o código seja robusto, confiável e atenda aos requisitos esperados. Existem várias abordagens e ferramentas pelas quais é possível realizar testes, proporcionando uma cobertura completa das funcionalidades do programa.

As **assertions** são utilizadas para verificar condições essenciais durante a execução do código. São cruciais para a detecção precoce de bugs, bem como para a garantia da consistência do código.

Os **doctests** oferecem uma abordagem única, incorporando testes diretamente na documentação do código-fonte. Isso não apenas fornece exemplos de uso atualizados na documentação, mas

também serve como uma verificação automatizada para assegurar que os exemplos apresentados estejam corretos.

O **unittest** é uma estrutura mais abrangente para testes em Python. Ele permite a organização de testes em classes e métodos, facilitando a manutenção e a execução seletiva de testes. As **assertions** fornecidas pelo **unittest** são mais robustas, oferecendo uma ampla gama de verificações. Esse mecanismo é ideal para projetos maiores, nos quais uma estrutura de teste mais formal é necessária para garantir a qualidade do código.

Para ter domínio sobre os testes que serão estudados nesta aula, suponha que você precise criar uma função que calcule a soma de uma lista de número e, em seguida, deva aplicar os três testes mencionados para saber se a função está correta.

Vamos Começar!

Assertions

As **assertions** são expressões utilizadas para verificar as condições de verdade durante a execução do código. Elas são fundamentais para a detecção precoce de erros, assegurando que as suposições sobre o comportamento do programa sejam atendidas. Confira, a seguir, um exemplo simples de como as assertions podem ser empregadas:

```
def divide(x, y):  
    assert y != 0,  
    return x / y  
  
result = divide(6, 0)  
print(result) # AssertionError: Divisão por zero!
```

O código cria uma função divisão, a qual, como bem sabemos, tem uma particularidade: a divisão por zero. Nesse caso, o algoritmo avisa sobre o erro.

Doctests

O **doctest** é um módulo em Python que permite incorporar testes diretamente na documentação do código, aproveitando os exemplos presentes na documentação para verificar se o código funciona conforme o esperado. Vamos considerar o seguinte trecho de código com doctests:

```
def square(x):
```

Retorna o quadrado de um número.

Exemplos:

```
>>> square(3)
```

```
9
```

```
>>> square(-2)
```

```
4
```

```
>>> square(0)
```

```
0
```

```
"""
```

```
    return x * x
```

```
import doctest
```

```
doctest.testmod()
```

```
TestResults(failed=0, attempted=3)
```

A função `square` é acompanhada por uma string de documentação que inclui exemplos de uso. Esses exemplos estão formatados de maneira especial, usando o prompt `>>>`, que indica um bloco de código Python.

Quando você executa `doctest.testmod()`, o módulo `doctest` procura todas as strings de documentação no seu código que contenham blocos de código `>>>` e executa esses blocos como testes. Ele compara a saída real desses blocos com o resultado esperado fornecido nos comentários.

No caso do exemplo anterior, `doctest.testmod()` executará a função `square(3)` e verificará se o resultado é igual a 9, executará `square(-2)` e verificará se o resultado é 4, e assim por diante.

Se todos os testes passarem, o `doctest` não produzirá nenhuma saída. Se houver uma discrepância entre a saída real e a esperada, o `doctest` imprimirá uma mensagem indicando onde ocorreu o problema.

A principal vantagem do `doctest` é que ele permite que você mantenha exemplos na documentação e, ao mesmo tempo, os utilize como testes automatizados. Isso ajuda a garantir que a documentação esteja sempre em sincronia com o código real. Além disso, quando você executa os testes, obtém uma validação automática dos exemplos apresentados na documentação.

Siga em Frente...

Módulo `unittest`

O módulo unittest disponibiliza uma estrutura de teste mais avançada, viabilizando a organização de testes em classes e métodos, além de fornecer assertions mais poderosas. Confira, a seguir, um exemplo simples de uso do unittest:

```
import unittest

def add(a, b):
    return a + b

class TestAddition(unittest.TestCase):
    def test_add_positive_numbers(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative_numbers(self):
        self.assertEqual(add(-2, -3), -5)

if __name__ == '__main__':
    import unittest
    unittest.main(argv=['first-arg-is-ignored'], exit=False)
    print()
```

A função add simplesmente soma dois números e cria uma classe que herda de unittest.TestCase. Isso indica que essa classe contém testes unitários. Dentro da classe de teste, você define métodos de teste. Cada método de teste deve começar com a palavra-chave test. Dentro desses métodos, você usa assertivas (como self.assertEqual) para verificar se o comportamento esperado do código é atendido. A condição if __name__ == '__main__': garante que a suíte de testes seja executada somente se o script for executado diretamente (não se for importado como um módulo em outro script). unittest.main() executa todos os testes definidos na classe TestAddition.

Vamos Exercitar?

Vamos pensar no problema apresentado no início desta aula. Para isso, considere uma função que calcula a soma de uma lista de números.

```
#assert
def sum_numbers(numbers):
    assert sum([1, 2, 3, 4]) == 10
    assert sum([-1, 0, 1]) == 0
    assert sum([]) == 0
    return sum(numbers)
```

```
teste = sum_numbers([1, 2, 3, 5])
print(teste)

#doctest
def sum_numbers(numbers):

    Soma os números em uma lista.

    Exemplos:
    >>> sum_numbers([1, 2, 3, 4])
    10
    >>> sum_numbers([-1, 0, 1])
    0
    >>> sum_numbers([])
    0
    """
    return sum(numbers)

if __name__ == :
    import doctest
    doctest.testmod()

#unittest
import unittest
def sum_numbers(numbers):
    return sum(numbers)

class TestSumNumbers(unittest.TestCase):
    def test_sum_numbers_positive(self):
        self.assertEqual(sum_numbers([1, 2, 3, 4]), 10)

    def test_sum_numbers_mixed(self):
        self.assertEqual(sum_numbers([-1, 0, 1]), 0)

    def test_sum_numbers_empty(self):
        self.assertEqual(sum_numbers([]), 0)

if __name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False)
```

Nesse código, criamos a função `sum_number`. A ideia é testar os três tipos de teste que estudamos nesta etapa de aprendizagem e aplicá-los ao caso investigado. Observe que cada situação foi resolvida de um modo distinto. O importante é sempre analisar qual a melhor solução para o problema em que estamos atuando.

Espero que você tenha gostado da solução! Lembre-se de que a prática é importante! Mude alguma parte do código e diversifique seu conhecimento!

Saiba mais

1. Para obter mais informações sobre testes com [Python](#), a leitura da documentação do Python é essencial.

2. Como mencionado anteriormente, uma leitura interessante para quem está começando a programar em Python é a do livro [Python 3: conceitos e aplicações: uma abordagem didática](#).

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book.

3. Outra dica para estudo e aprofundamento sobre esse tema é o livro [Use a cabeça! Python](#).

BARRY, P. **Use a cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book.

Referências

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/books/9788536530253>. Acesso em: 21 out. 2023.

BARRY, P. **Use a cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786555207842>. Acesso em: 12 out. 2023.

LEMBURG, M. PEP 249 - Python database API specification v.2.0. **Python Enhancement Proposals**, 12 abr. 1999. Disponível em: <https://peps.python.org/pep-0249/#description>. Acesso em: 31 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29. ed. São Paulo: Érica, 2019.

UNITTEST – Unit testing framework. **Python 3.12.2 Documentation**, [s. d.]. Disponível em: <https://docs.python.org/3/library/unittest.html>. Acesso em: 16 nov. 2023.

Aula 4

Machine Learning com Python

Machine learning com Python

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

Machine learning (ML) é um campo da inteligência artificial que se concentra no desenvolvimento de algoritmos e modelos que permitem que um sistema aprenda padrões a partir de dados e faça previsões ou tome decisões mesmo que não seja explicitamente programado para tal. Em vez de seguir instruções específicas, os modelos de ML são alimentados com dados e ajustam seus parâmetros para que se adaptem da melhor maneira possível a esses dados. Tal método de aprendizado possibilita que os modelos generalizem e tomem decisões em novos dados.

No processo de machine learning, o treinamento é crucial para permitir que o modelo aprenda padrões com base nos dados. Existem vários tipos de treinamento, cada um com características distintas.

TensorFlow é uma biblioteca de código aberto desenvolvida pela Google que facilita a implementação de modelos de machine learning e deep learning.

Sua estrutura flexível viabiliza a criação e o treinamento de modelos complexos, sendo amplamente utilizada na comunidade de aprendizado de máquina.

Para contextualizar sua aprendizagem, imagine que você seja o gerente de uma pequena loja que vende produtos exclusivos. Ao analisar o histórico de vendas ao longo de um ano, você percebeu que as vendas variam à medida que os meses se passam. Para tomar decisões mais informadas

sobre o estoque e as estratégias de marketing, você opta por explorar a possibilidade de prever as vendas futuras.

Vamos Começar!

Teoria de machine learning

Machine learning (ML) – Aprendizado de Máquina, em português – é um campo da inteligência artificial que se concentra no desenvolvimento de algoritmos e modelos capazes de aprender padrões a partir de dados. O propósito central do ML é capacitar computadores para que realizem tarefas específicas mesmo que não sejam explicitamente programados para tais ações, permitindo que eles evoluam e melhorem com a experiência. O processo de aprendizado envolve a exposição a conjuntos de dados, nos quais o sistema identifica padrões, correlações e regras, preparando-se para realizar previsões ou tomar decisões em novos dados.

Existem diversas abordagens no campo de ML, como: aprendizado supervisionado, em que o modelo é treinado com dados rotulados; aprendizado não supervisionado, no qual o modelo busca por padrões em dados não rotulados; e o aprendizado por reforço, em que o modelo interage com um ambiente dinâmico, aprendendo com as recompensas obtidas. O machine learning tem aplicações abrangentes, que envolvem desde o reconhecimento de imagens até a tomada de decisões complexas em setores como finanças e saúde. Trata-se de um campo em constante evolução, impulsionado por avanços tecnológicos e pela crescente disponibilidade de dados, tornando-se uma ferramenta essencial em diversas áreas.

Alguns dos modelos e algoritmos de ML mais utilizados são:

1. Árvores de decisão: modelo que toma decisões com base em condições.
2. Redes neurais: inspiradas no funcionamento do cérebro, são usadas para problemas complexos.
3. Support Vector Machine (SVM): usado para classificação e regressão.
4. K-Means: algoritmo de agrupamento utilizado no aprendizado não supervisionado.

Tipos de treinamento

Os tipos de treinamento são estratégias fundamentais no desenvolvimento de modelos de machine learning, pois determinam como o algoritmo aprenderá com os dados disponíveis. Cada abordagem tem características distintas e aplicações específicas, proporcionando flexibilidade para diferentes contextos.

No treinamento supervisionado, o modelo é alimentado com um conjunto de dados rotulado, que consiste em pares de entrada e saída esperadas. O objetivo é que o modelo aprenda a mapear entradas para saídas corretas. Um exemplo prático pode ser observado na classificação de e-mails como “spam” ou “não spam”.

Ao contrário do supervisionado, o treinamento não supervisionado lida com dados não rotulados. O modelo é desafiado a encontrar padrões e estruturas intrínsecas aos dados sem informações prévias sobre as saídas. Como exemplo prático, podemos citar o agrupamento de clientes com base em padrões de compra.

O treinamento por reforço envolve um agente interagindo com um ambiente dinâmico. O modelo aprende a realizar ações que maximizem as recompensas ao longo do tempo. Exemplo prático: treinar um agente de IA para jogar um jogo e ganhar pontos ao realizar as ações corretas.

Siga em Frente...

TensorFlow

TensorFlow é uma biblioteca de código aberto desenvolvida pela Google que facilita a implementação de modelos de machine learning e deep learning. Sua estrutura flexível permite a criação e o treinamento de modelos complexos, sendo amplamente utilizada na comunidade de aprendizado de máquina.

A seguir, vamos analisar três exemplos dessa biblioteca: uma para treinamento supervisionado, outro para treinamento não supervisionado e, por fim, um para treinamento por reforço.

Supervisionado

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Dados de exemplo
X_train = tf.constant([[1.0], [2.0], [3.0], [4.0]])
y_train = tf.constant([[2.0], [4.0], [6.0], [8.0]])

# Modelo de Regressão Linear Simples
model = Sequential()
model.add(Dense(units=1, input_shape=(1,)))
model.compile(optimizer='sgd', loss='mean_squared_error')
```

```
# Treinamento do modelo
model.fit(X_train, y_train, epochs=1000, verbose=0)

# Previsão
X_new = tf.constant([[5.0]])
prediction = model.predict(X_new)
print("Predição:", prediction[0][0])
plt.ylabel('Notas')
plt.show()
```

Não supervisionado

```
import tensorflow as tf
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model

# Dados de exemplo
X_unsupervised = tf.constant([[1.0, 2.0], [2.0, 3.0], [3.0, 4.0], [4.0, 5.0]])

# Modelo Autoencoder Simples
input_layer = Input(shape=(2,))
encoded = Dense(units=1)(input_layer)
decoded = Dense(units=2)(encoded)

autoencoder = Model(inputs=input_layer, outputs=decoded)
autoencoder.compile(optimizer='adam', loss='mean_squared_error')

# Treinamento do modelo não supervisionado
autoencoder.fit(X_unsupervised, X_unsupervised, epochs=1000, verbose=0)

# Previsão
prediction_unsupervised = autoencoder.predict(X_unsupervised)
print("Predição Não Supervisionada:", prediction_unsupervised)
```

Por reforço

```
import tensorflow as tf
import gym

# Ambiente CartPole do Gym
env = gym.make('CartPole-v1')
```

```
# Modelo Simples para Aprendizado por Reforço
model_reinforcement = tf.keras.Sequential([
    tf.keras.layers.Dense(24, activation='relu', input_shape=(env.observation_space.shape[0],)),
    tf.keras.layers.Dense(env.action_space.n, activation='linear')
])

model_reinforcement.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
loss='mse')

# Treinamento por Reforço (exemplo fictício)
max_episodes = 1000 # Defina o número máximo de episódios
for episode in range(max_episodes):
    state = env.reset()
    done = False
    while not done:
        action = env.action_space.sample()
        next_state, reward, done, _ = env.step(action)
        target = reward + 0.95 *
tf.reduce_max(model_reinforcement.predict(next_state.reshape(1, -1)))
        target_f = model_reinforcement.predict(state.reshape(1, -1))
        target_f[0][action] = target
        model_reinforcement.fit(state.reshape(1, -1), target_f, epochs=1, verbose=0)
        state = next_state

# Condição de parada
if episode % 10 == 0:
    average_reward = sum(reward for _ in range(10)) / 10.0
    print(f'Episode {episode}, Average Reward: {average_reward}')

# Adicionando uma condição de parada
if average_reward == 1: # Pode ajustar esse valor conforme necessário
    print(f'Solved after {episode} episodes!')
    break
```

Vamos Exercitar?

Vamos pensar no problema apresentado no início desta aula. Para isso, criaremos um modelo de previsão.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf

# Crie dados fictícios de vendas ao longo do tempo
np.random.seed(42)
meses = np.arange(1, 13)
vendas = np.array([200, 220, 250, 280, 300, 320, 350, 380, 400, 420, 450, 480])

# Crie um DataFrame
dados = pd.DataFrame({'Mes': meses, 'Vendas': vendas})

# Visualize os dados
plt.scatter(dados['Mes'], dados['Vendas'])
plt.xlabel('Mês')
plt.ylabel('Vendas')
plt.title('Dados de Vendas ao Longo do Tempo')
plt.show()

# Normalização dos dados de treinamento
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Divida os dados em conjunto de treinamento e teste
X = dados[['Mes']]
y = dados['Vendas']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Crie e treine o modelo de regressão linear usando TensorFlow
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1,)), # Camada de entrada
    tf.keras.layers.Dense(units=8, activation='relu'), # Camada escondida com ativação ReLU
    tf.keras.layers.Dense(units=1) # Camada de saída
])

model.compile(optimizer='adam', loss='mean_squared_error')

# Treine o modelo por mais épocas
model.fit(X_train, y_train, epochs=500, verbose=0)
```

```
# Faça previsões no conjunto de teste
predictions = model.predict(X_test)

# Desfaça a normalização para avaliar o desempenho
min_sales = dados['Vendas'].min()
max_sales = dados['Vendas'].max()

predictions_inverse = predictions * (max_sales - min_sales) + min_sales
y_test_inverse = y_test * (max_sales - min_sales) + min_sales

# Visualize as previsões em relação aos dados reais
plt.scatter(X_test, y_test_inverse, label='Dados Reais')
plt.plot(X_test, predictions_inverse, color='red', label='Previsões')
plt.xlabel('Mês')
plt.ylabel('Vendas')
plt.title('Previsões de Vendas com Regressão Linear (TensorFlow)')
plt.legend()
plt.show()

# Avalie o desempenho do modelo
erro_mse = mean_squared_error(y_test_inverse, predictions_inverse)
print(f'Erro Médio Quadrático (MSE): {erro_mse:.2f}')

# Faça uma previsão para o próximo mês
proximo_mes_scaled = scaler.transform(np.array([[13]]))
previsao_proximo_mes_scaled = model.predict(proximo_mes_scaled)
previsao_proximo_mes = scaler.inverse_transform(previsao_proximo_mes_scaled)[0, 0]
print(f'Previsão de Vendas para o Próximo Mês: {previsao_proximo_mes:.2f}')
```

Utilizamos um modelo de treinamento supervisionado e criamos a previsão de venda para o próximo ano. Lembre-se de que para esse caso empregamos dados fictícios, porém a ideia a ser considerada para o desenvolvimento de outras construções é semelhante.

Espero que você tenha gostado da solução! Lembre-se de que a prática é importante! Mude alguma parte do código e diversifique seu conhecimento!

Saiba mais

1. Na obra indicada para leitura a seguir, você terá contato com os primeiros conceitos de ciência de dados (data science) e ainda reforçará seu conhecimento sobre machine learning. Para acessar o conteúdo recomendado, clique no link a seguir.

AMÍLCAR NETTO; MACIEL, F. [Python para data science e machine learning descomplicado](#). Rio de Janeiro: Alta Books, 2021. E-book.

2. Como mencionado anteriormente, uma leitura interessante para quem está começando a programar em Python é a do livro [Python 3: conceitos e aplicações: uma abordagem didática](#).

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book.

3. Também encorajo você a conhecer a comunidade do TensorFlow, que, como já disse, é altamente rica em informações e aplicações dessa biblioteca. Para saber mais detalhes, visite: [TensorFlow](#).

Referências

AMÍLCAR NETTO; MACIEL, F. **Python para data science e machine learning descomplicado**. Rio de Janeiro: Alta Books, 2021. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786555203172/>. Acesso em: 21 out. 2023.

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/books/9788536530253>. Acesso em: 21 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29. ed. São Paulo: Érica, 2019.

TENSORFLOW. **Página inicial**, [s. d.]. Disponível em: <https://www.tensorflow.org/?hl=pt-br>. Acesso em: 16 nov. 2023.

Aula 5

Aplicações com Python

Videoaula de Encerramento

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Chegada

Olá, estudante! Para desenvolver a competência associada a esta unidade de aprendizagem, que é **"Diferenciar contextos distintos de utilização da linguagem de programação Python"**, devemos, antes de tudo, conhecer as principais aplicações de Python. Além disso, é importante aprender os passos a serem seguidos em cada aplicação, seja no desenvolvimento web, seja no machine learning.

Ao longo desta etapa de estudos, foi possível aliar os assuntos estudados à prática de diversas formas. Mostrou-se a diferença entre front-end e back-end no desenvolvimento web, conceitos que são utilizados por muitas outras linguagens (Maciel, 2018).

Ao examinar as possíveis aplicações de Python, pudemos aprofundar nosso entendimento sobre o desenvolvimento mobile (Morais *et al.*, 2022), segmento no qual o Python vem ganhando notoriedade, assim como já acontece no desenvolvimento web. Quando programamos, seja qual for a linguagem adotada, os testes de funcionamento se tornam importantes para validar resultados e criar aplicações corretas. Nesse contexto, aprendemos especificamente sobre assertions, doctests e unittest.

Em relação ao machine learning, a biblioteca TensorFlow merece destaque, visto que, além de suas muitas funcionalidades, a constante contribuição da comunidade Python vinculada a essa biblioteca torna a sua utilização bem relevante (TensorFlow, [s. d.]).

Durante este percurso de aprendizagem, você não apenas assimilou os princípios e técnicas apresentados, mas também os aplicou a situações do mundo real. Saber utilizar a linguagem Python nos diversos cenários apresentados em nossos estudos é o que faz de nós, programadores de Python, um recurso importante e desejado pelo mercado. Logo, a necessidade de se manter atualizado é grande.

Essa gama de aptidões o habilita para se transformar em um resolvidor experiente de desafios tecnológicos e computacionais, conferindo-lhe a capacidade de abordar questões de forma eficiente. Além disso, adquirimos proficiência no emprego de aplicações que podem envolver Python.

É Hora de Praticar!



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Vamos criar um classificador de dígitos escritos à mão, considerando os números de 0 a 9. Para tanto, usaremos um banco de dados pronto do TensorFlow. Em resumo, dada a imagem de um número escrito à mão, quão eficiente será o modelo para acertar o número?

Questões norteadoras:

1. Como você pode aplicar seus conhecimentos em programação em Python para criar esse modelo?
2. Como é possível ajustar os parâmetros do treino do modelo para torná-lo melhor, caso seja necessário?

Para encerrar e consolidar seu aprendizado, reflita sobre as seguintes perguntas:

1. Aprender sobre as diversas aplicações da linguagem Python me ajuda a ser um programador mais completo?
2. Qual é a importância das bibliotecas “prontas” da Python tanto no contexto de aplicação na web quanto para mobile?
3. Como a biblioteca TensorFlow ajuda a construir modelos de machine learning mais eficientes e robustos?

Essas considerações ajudarão você a incorporar de maneira mais profunda o conhecimento adquirido e a compreender o alcance de suas aplicações. Desejo a você muito sucesso em sua jornada de aprendizagem!

```
import tensorflow as tf
mnist = tf.keras.datasets.mnist
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5)
model.evaluate(x_test, y_test)
```

O conjunto de dados MNIST é um conjunto de imagens de dígitos escritos à mão (0 a 9). É frequentemente usado como um ponto de partida para o aprendizado de máquina. `x_train` e `x_test` contêm as imagens, enquanto `y_train` e `y_test` contêm as etiquetas correspondentes.

As imagens são normalizadas dividindo-se cada pixel por 255.0. Isso coloca os valores dos pixels no intervalo [0, 1], facilitando o treinamento do modelo.

Cria-se, então, um modelo sequencial com as seguintes camadas:

Flatten: converte a matriz bidimensional das imagens (28x28 pixels) em um vetor unidimensional.

Dense(128, activation='relu'): camada densa com 128 neurônios e função de ativação ReLU.

Dropout(0.2): regularização por abandono, desativando aleatoriamente 20% dos neurônios durante o treinamento, para evitar overfitting.

Dense(10, activation='softmax'): camada de saída com 10 neurônios (um para cada dígito) e função de ativação softmax, que produzirá uma distribuição de probabilidade sobre as classes.

O modelo é compilado com o otimizador 'adam', a função de perda 'sparse_categorical_crossentropy' (adequada para problemas de classificação multiclasse) e a métrica de 'accuracy'.

O modelo é treinado usando-se o conjunto de treinamento (`x_train` e `y_train`) por cinco épocas.

O desempenho do modelo é avaliado utilizando-se o conjunto de teste (`x_test` e `y_test`), e os resultados, incluindo a perda e a precisão, são exibidos.

A arquitetura específica do modelo envolve uma camada inicial que “achata” as imagens bidimensionais em um vetor unidimensional, seguida por uma camada densa (totalmente conectada) com ativação ReLU, uma camada de dropout para regularização e, por fim, uma camada de saída com ativação softmax para gerar probabilidades a cada classe.

Após o treinamento, o modelo pode ser usado para prever a classe de um dígito desconhecido, e a avaliação no conjunto de teste fornece uma medida de quão bem ele generaliza para dados não vistos. A métrica de “accuracy” indica a proporção de previsões corretas em relação ao total de previsões.

O material visual a seguir esquematiza os principais tópicos abordados nesta unidade de aprendizagem em que tratamos de aplicações em Python. Este infográfico exibe uma percepção clara e sucinta de cada parte desta etapa de estudos, enfatizando os conceitos e fundamentos necessários para uma boa compreensão dos saberes desenvolvidos.

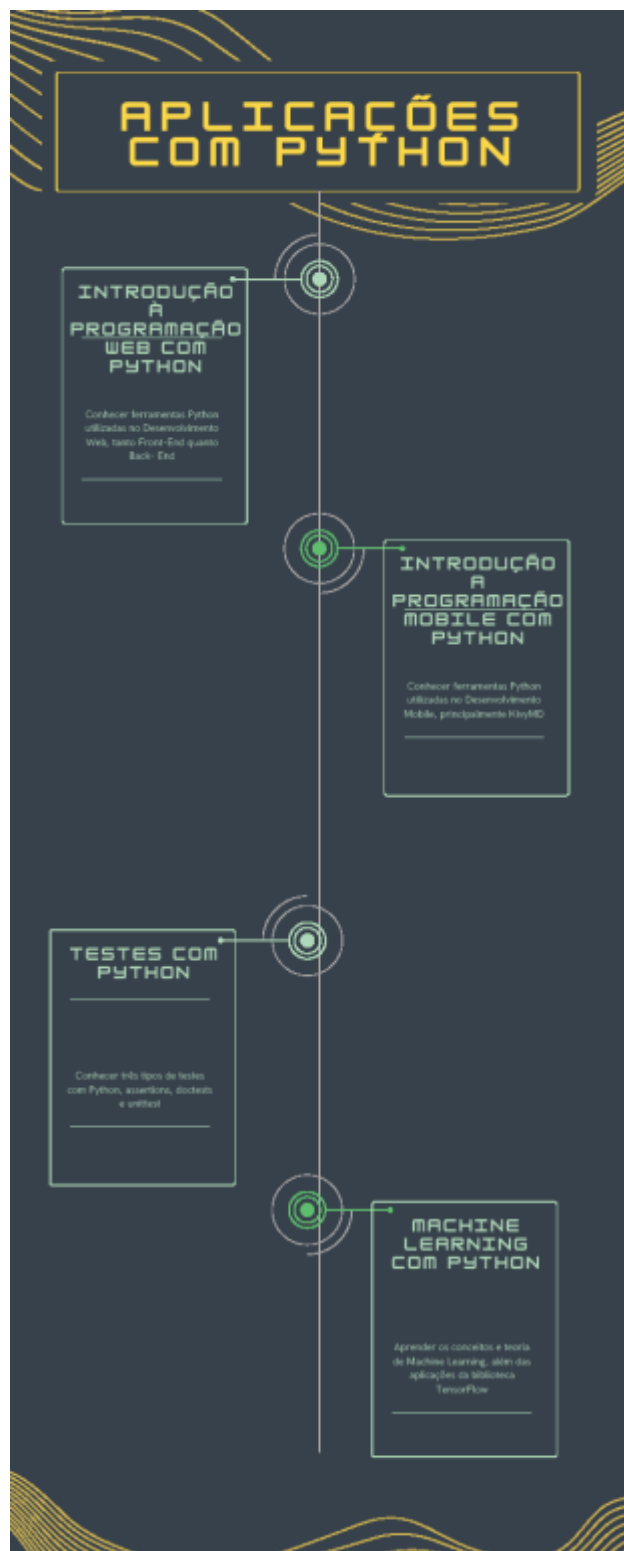


Figura 1 | Infográfico: aplicações com Python. Fonte: elaborada pelo autor.

AMÍLCAR NETTO; MACIEL, F. **Python para data science e machine learning descomplicado**. Rio de Janeiro: Alta Books, 2021. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786555203172/>. Acesso em: 21 out. 2023.

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/books/9788536530253>. Acesso em: 21 out. 2023.

LEMBURG, M. PEP 249 - Python database API specification v.2.0. **Python Enhancement Proposals**, 12 abr. 1999. Disponível em: <https://peps.python.org/pep-0249/#description>. Acesso em: 31 out. 2023.

MACIEL, F. M. B. **Python e Django: desenvolvimento web moderno e ágil**. Rio de Janeiro: Alta Book, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786555200973>. Acesso em: 21 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29. ed. São Paulo: Érica, 2019.

MORAIS, M. S. de F. *et al.* **Fundamentos de desenvolvimento mobile**. Porto Alegre: Grupo A, 2022. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786556903057>. Acesso em: 21 out. 2023.

PYTHON para desenvolvimento mobile. **Python Brasil**, 3 jun. 2016. Disponível em: <https://python.org.br/mobile/>. Acesso em: 15 nov. 2023.

TENSORFLOW. **Página inicial**, [s. d.]. Disponível em: <https://www.tensorflow.org/?hl=pt-br>. Acesso em: 16 nov. 2023.

UNITTEST – Unit testing framework. **Python 3.12.2 Documentation**, [s. d.]. Disponível em: <https://docs.python.org/3/library/unittest.html>. Acesso em: 16 nov. 2023.